A Report on

"Malware Analysis & Threat Attribution Using TTPs and CVSS-Based Mapping"

Submitted in partial fulfilment for the Degree of

B TECH-M TECH INTEGRATED

In

CYBER SECURITY

Submitted By

NENAVATH POOJA

(101CTBMCS2122027)

Under the Supervision of

Mr. Prakash Khasor

(Assistant Professor)

Submitted to

SCHOOL OF CYBER SECURITY & DIGITAL FORENSICS

NATIONAL FORENSIC SCIENCES UNIVERSITY

GANDHINAGAR – 382007, GUJARAT, INDIA.

[Dec, 2025]

DECLARATION By STUDENT

I hereby declare that the work being presented in this Report titled ***"Malware Analysis & Threat Attribution Using TTPs and CVSS-Based Mapping"*** *by me i.e.* ***Nenavath Pooja****, having Enrolment Number* ***"101CTBMCS2122027"****, and submitted to the School of Cyber Security and Digital Forensic at National Forensic Sciences University, Gandhinagar; is my original work carried out during the period of* ***"August 2025"*** *to* ***"Dec 2025"*** *under the supervision of* **"Mr. Prakash Khasor ".** *I have complied with the School's Ethical Code of Conduct and have duly acknowledged all sources through proper citations and references. The plagiarism check confirms that the overall similarity index is within 10%.*

_____

(Name & Signature of Student)

CERTIFICATE FROM GUIDE

*I hereby certify that the dissertation entitled "**Malware Analysis and Threat Attribution Using TTPs and CVSS-Based Mapping",** embodies the result of bonafide project work done by **Nenavath Pooja** of the Semester **X,** having enrollment number **101CTBMCS2122027** for the degree of **B tech-MtechCSE(Cyber Security)** in the "**School of Cyber Security & Digital Forensics**" of the "**National Forensic Sciences University, Gandhinagar, Gujarat – India**" under my guidance and supervision. I further certify that this is an original work and that whatever material obtained and used from other sources has been duly acknowledged in the report. This work has not been submitted for any degree or diploma of any university or institute.*

Date: 22/12/2025

Place: *Gandhinagar, Gujarat*

Mr. Prakash Khasor
Assistant Professor
School of Cyber Security & Digital Forensics
National Forensic Sciences University Gandhinagar,
Gujarat – India, 382007.

CERTIFICATE

This is to certify that ***Ms. Nenavath Pooja ,*** *of the Semester* ***X,*** *having enrollment number* ***"101CTBMCS2122027"*** *has submitted the thesis/dissertation entitled "****Malware Analysis & Threat Attribution Using TTPs and CVSS-Based Mapping****" for the partial fulfilment of the requirement for the award of the degree of "****B tech - Mtech CSE(Cyber Security)****" in the* ***School of Cyber Security & Digital Forensics*** *at* ***National Forensic Sciences University, Gandhinagar, Gujarat - India****.*

Signature of Supervisor
Mr. Prakash Khasor

Associate Dean / Dean
School of Cyber Security & Digital Forensics
National Forensic Sciences University
Gandhinagar, Gujarat – India, 382007.

This is to certify that the viva-voce of ***Ms. Nenavath Pooja*** *of the* ***School of Cyber Security & Digital Forensics, National Forensic Sciences University, Gandhinagar, Gujarat, India - 382007*** *was conducted on* ***22/12/2025*** *at the* ***National Forensic Sciences University, Gandhinagar Campus. His/Her*** *thesis /dissertation titled "****Malware Analysis &Threat Attribution Using TTPs and CVSS-Based Mapping****" was evaluated and found satisfactory.*

Name & Signature of Internal Examiner ***Name & Signature of External Examiner***

Date: *22/12/2025*
Place: *Gandhinagar, Gujarat*

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed significantly to the successful completion of my dissertation titled **"Malware Analysis & Threat Attribution Using TTPs and CVSS-Based Mapping."** The journey of completing this project was both challenging and rewarding, and it would not have been possible without the guidance, support, and encouragement of several individuals.

First and foremost, I owe my deepest gratitude to **Mr. Prakash Khasor**, my project supervisor, for his invaluable guidance, constant motivation, and insightful feedback throughout the course of this work. His vast knowledge, patience, and constructive suggestions played a crucial role in shaping this project and enhancing its overall quality. I am truly thankful for the time and effort he dedicated to mentoring me at every stage of the dissertation.

I would also like to extend my appreciation to my friends for their continuous support and for contributing valuable ideas during discussions on the major challenges encountered in the project. Their constructive feedback and collaborative approach helped in problem-solving and made the research process more engaging and effective.

With Sincere Regards,

Nenavath Pooja

B tech- M tech(CSE) cyber security

ABSTRACT

The rapid evolution of malware, particularly ransomware and polymorphic threats, has rendered traditional signature-based detection mechanisms increasingly ineffective. Security analysts face significant challenges in timely threat triage and attribution due to the sheer volume of alerts and the complexity of modern attack vectors. This project proposes a comprehensive, automated framework for Malware Analysis and Threat Attribution that integrates static code analysis with dynamic behavioural monitoring.

The proposed system includes a multi-layered detection engine: a Static Analyzer that scrutinizes Windows Portable Executable (PE) files to extract Indicators of Compromise (IOCs), API import anomalies, and Shannon entropy metrics; and a Dynamic Behavioural Agent (Ransom Guard), which monitors real-time file system events to detect heuristic ransomware patterns such as rapid file encryption and mass modification. Uniquely, the system bridges the gap between raw data and actionable intelligence by automatically mapping observed behaviours to the MITRE ATT&CK framework (Tactics, Techniques, and Procedures) and correlating vulnerabilities with CVSS (Common Vulnerability Scoring System) risk scores.

The solution is implemented as a unified web-based dashboard using Streamlit, enabling analysts to visualize threat scores, inspect attack lifecycles, and generate automated forensic reports. Experimental results demonstrate the system's efficacy in identifying malicious samples with high entropy and suspicious API calls, providing a robust, low-latency tool for early threat detection and incident response.

Keywords: Malware Analysis, Ransomware Detection, MITRE ATT&CK, Static Analysis, Dynamic Monitoring, Cybersecurity, Threat Attribution.

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
AV	Antivirus
CPU	Central Processing Unit
CVE	Common Vulnerability and Exposures
CVSS	Common Vulnerability Scoring system
DLL	Dynamic Link Library
HTTPs	Hypertext Transfer Protocol Secure
IAT	Import Address Table
IDS	Intrusion Detection system
IOC	Indicators of Compromise
MD5	Message Digest System
MITRE	Massachusetts Institute of Technology Research & Engineering
NLP	Natural Language Processing
OS	Operating system
PC	Packet capture
PE	Portable Executable
SHA	Secure Hash Algorithm
SIEM	Security Information & Event Management
TTP	Tactics, Techniques, and Procedures

LIST OF TABLES

Tables	Pages No.
Behavioural Indicators	16
Mitre Mapping	16
Static Analysis	21
Behavioural Threshold	27
Threat	27
Comparative Analysis with Traditional AV	30

LIST OF FIGURES

Figure No.	Names.	Pages No.
Fig 1	System Architecture	15
Fig 2	Code Execute	21
Fig 3	Malware Analysis and Threat Attribution	22
Fig 4	Static Analysis	23
Fig 5	Static Analysis TTPs	23
Fig 6	TTPs Technique	24
Fig 7	IOC Details	24
Fig 8	Suspicious Malware of the Technique	25
Fig 9	Technique Details	25
Fig 10	Technique Details with Detection and Mitigations	26
Fig 11	ML Classifier	28
Fig 12	Models Details	29
Fig 13	Model Metrics	29
Fig 14	Manual Prediction	30

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

LIST OF TABLES

LIST OF FIGURES

1.Introduction……. 12

1.1 Motivation……. 12

1.2 Scope of the Project…….. 12

1.3Organization of Report…….. 13

2. Theoretical Background & Literature Survey……. 14

2.1ComparativeAnalysis of Existing Schemes…… 14

2.2 The MITRE ATT&CK Framework…….. 14

2.3 CVSS & Vulnerability Risk Scoring……. 15

3. The Proposed Model and Implementation Methodology……16

3.1The Problem Statement…… 16

3.2 The System Model OR Sketch of the Framework……16

3.3Methodology …… 16

4. Result & Analysis…… 19

4.1 Experimental Setup ……..… 19

4.2 Evaluation Metrics …… 20

4.3 Static Analysis Results……. 21

4.4 Dynamic Analysis analysis …… 26

4.5 Threat Attribution & Mitre Mapping Results……27

4.6 Scenario Based Analysis……27

4.7 Comparative Analysis With Traditional AV……29

4.8 Limitations Observed…… 29

4.9 Overall Result Interpretation…… 30

5. Conclusion……..32

6. Future Work and Plan……33

APPENDIX……..34

REFERENCES…… 39

Chapter 1: INTRODUCTION

Purpose

The primary purpose of this project is to develop an automated, unified framework for analysing modern malware threats, designed to overcome the limitations of traditional signature-based detection systems. By integrating static code analysis with real-time behavioural monitoring, the system identifies malicious activities such as ransomware encryption and suspicious API calls based on operational heuristics rather than just known signatures. Furthermore, the project aims to streamline the incident response process by automatically mapping observed behaviours to the MITRE ATT&CK framework and CVSS vulnerability scores, providing security analysts with immediate, actionable intelligence and comprehensive forensic reports for rapid threat attribution and remediation

1.1 Motivation

In the modern cybersecurity landscape, malware, specifically ransomware has evolved from simple pranks to sophisticated cyber-espionage and extortion tools (e.g., WannaCry, Ryuk). Traditional antivirus solutions rely heavily on signature-based detection, which fails to identify zero-day attacks or polymorphic malware that changes its code signature. There is a critical need for an automated system that not only detects malware but also attributes the threat by analysing its behaviour (TTPs - Tactics, Techniques, and Procedures) and potential exploited vulnerabilities (CVEs). This project addresses this gap by combining static analysis, dynamic behavioural monitoring, and standardized threat mapping (MITRE ATT&CK/CVSS) into a unified dashboard.

1.2 Scope of the Project

The project focuses on the automated analysis of Windows Portable Executable (PE) files and associated vulnerability data. The scope includes:

Static Analysis: Analysing executable files without running them to extract metadata, hashes, and code features.

Dynamic Analysis: Monitoring real-time system behaviour on a victim interface to detect ransomware-like activities (e.g., mass file encryption).

Threat Attribution: Mapping observed behaviours and APIs to the MITRE ATT&CK framework to identify adversarial tactics.

Vulnerability Assessment: Fetching and scoring CVEs using the Common Vulnerability Scoring System (CVSS). Reporting: Generating professional PDF reports for security analysts.

1.3 Organization of Report

The report is organized as follows:

Section 1 covers the theoretical backbone including MITRE and CVSS.

Section 2 details the proposed system architecture and methodology.

Section 3 discusses the implementation results and analysis metrics.

Section 4 concludes the findings, and

Section 5 outlines future enhancements.

Chapter 2 Theoretical Background & Literature Survey

2.1 Comparative Analysis of Existing Schemes Signature-Based Detection:

Comparative Analysis of Existing Malware Detection Paradigms To understand the necessity of the proposed framework; it is essential to analyse the strengths and limitations of current malware detection methodologies.

Signature-Based Detection: This traditional method matches files against a database of known byte patterns or hashes (e.g., MD5/SHA256). While it is extremely fast and accurate for identifying known threats, it is completely ineffective against obfuscated, polymorphic, or zero-day malware that has not been seen before.

Heuristic-Based Detection: This approach uses static rules and algorithmic assumptions to scan for suspicious logic within the code. Although it can detect new variants of malware, it is highly susceptible to false positives, often flagging legitimate software as malicious.

Sandbox-Based Dynamic Analysis: This involves executing the malware in a secured, isolated virtual environment to capture its runtime behaviour. However, modern malware is increasingly "environment-aware," capable of detecting when it is running inside a sandbox and suppressing its malicious behaviour to evade detection. Behaviour-Centric Threat Attribution (Proposed Approach): The proposed model focuses on observing what the malware naturally does (e.g., mass file modification, high entropy generation) rather than just examining its code structure. By correlating these behaviours with known attacker playbooks, this approach generates actionable intelligence regardless of code obfuscation.

2.2 The MITRE ATT&CK Framework

The MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework is a globally recognized knowledge base that documents real-world attacker behaviours. Unlike traditional classification that labels malware generically (e.g., "Trojan"), MITRE categorizes actions into:

Tactics: The technical goals of the attacker (the "Why"). Example: Impact, Defence Evasion, Persistence. Techniques: The specific methods used to achieve those goals (the "How").

Example: File Encryption (Technique) is used to achieve Impact (Tactic). Implementing this framework allows security teams to move beyond simple detection. It enables them to understand specific attack objectives, align detection mechanisms with global threat intelligence, and significantly improve the prioritization of incident response efforts.

2.3 CVSS & Vulnerability Risk Scoring

The Common Vulnerability Scoring System (CVSS) is an open industry standard for assessing the severity of computer system security vulnerabilities. It provides a numerical score ranging from 0.0 to 10.0, calculated based on several metrics:

Attack Complexity: How difficult it is to exploit the vulnerability. **Privileges Required:** The level of access an attacker needs. **Impact Metrics:** The potential consequence to the Confidentiality, Integrity, and Availability (CIA triad) of the system. By integrating CVSS into this project, the system links observed malware behaviours with potential exploit paths. This quantification allows analysts to objectively prioritize remediation tasks based on calculated risk rather than intuition.

Chapter 3 Proposed Model & Methodology

3.1 Problem Statement

Security analysts face three key challenges:

1. Information overload from alerts
2. Lack of correlation between static and dynamic findings
3. No unified view of malware behaviour, exploited vulnerabilities, and attacker intent

This project aims to automate correlation, reduce analyst effort, and provide decision-ready intelligence

3.2 System Architecture

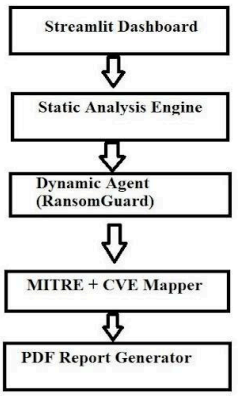


Fig: 1 System Architecture

3.3 Methodology

A. Static Analysis Module

This module inspects executables **without execution**, ensuring safety.

Key Analysis Components

1. **Cryptographic Hashing**

1.1 MD5: Quick identification

1.2 SHA-256: Collision-resistant tracking

2. **Entropy Analysis**

3. High entropy sections often indicate:

   Packing

   Encryption

   Obfuscation

4. **Import Address Table (IAT) Inspection**

   Identifies suspicious API dependencies

   Correlates APIs with attacker capabilities

5. **String Analysis**

   Extracts embedded indicators such as:

   - C2 URLs
   - Cryptocurrency wallet addresses
   - Ransom messages

B. Dynamic Analysis Module (RansomGuard)

This module continuously monitors runtime behaviour.

Behavioural Indicators

Indicator	Rationale
High file modification rate	Typical ransomware behaviour
Increased file entropy	Indicates encryption
Suspicious file extensions	Known ransomware patterns

Directory traversal	Targeting user data

Table:1

The system uses **event-driven callbacks**, ensuring minimal system overhead.

C. Threat Attribution Logic

MITRE Mapping Example

Observed Behaviour	Mapped Technique
CryptEncrypt API	T1486 – Data Encrypted for Impact
IsDebuggerPresent	T1497 – Sandbox Evasion
VirtualAllocEx	T1055 – Process Injection

Table: 2

Threat Scoring Formula

A weighted score combining:

1. Behavioural severity
2. Number of malicious indicators
3. Associated CVSS scores

Chapter 4 Results & Analysis

This section presents a comprehensive evaluation of the proposed Malware Analysis and Threat Attribution framework. Both **theoretical reasoning** and **empirical observations** are discussed to assess the effectiveness, accuracy, and practicality of the system. The results demonstrate how the integration of static analysis, dynamic behavioural monitoring, and standardized threat mapping provides deeper insights compared to conventional malware detection approaches.

4.1 Experimental Setup

To ensure realistic and controlled testing, the experiments were conducted in an **isolated virtualized environment**, preventing unintended system damage and external network propagation.

Hardware & Environment Configuration

1. **Host Machine**: Windows-based analysis server
2. **Victim Machine**: Windows 10 Virtual Machine
3. **Virtualization**: VMware / VirtualBox
4. **Network Mode**: Host-only (no internet access during execution)

Software & Tools Used

1. **Python Version**: Python 3.9+
2. **Static Analysis Libraries**:

   `pefile` – PE header and import analysis

   `hashlib` – cryptographic hash computation

3. **Dynamic Monitoring**:

   `watchdog` – real-time filesystem event tracking

4. **Visualization & Reporting**:

`Streamlit` – interactive dashboard

`Plotly` – charts and gauges

`FPDF` – automated PDF report generation

Dataset Description

1. **Malicious Samples**:

Known ransomware families (e.g., WannaCry-like behavior)

Packed and obfuscated executables

2. **Benign Samples**:

System utilities (Notepad++, Calculator)

Legitimate installers and compressed executables

This mixed dataset ensured that the system was evaluated against **both malicious and legitimate real-world binaries**.

4.2 Evaluation Metrics

To objectively measure system performance, multiple quantitative and qualitative metrics were defined.

1. Entropy Score

1. Measures randomness of file content

2. Scale: **0.0 – 8.0**

3. Interpretation:

`< 6.0` → Normal / Textual

`6.0 – 7.0` → Compressed / Packed

`> 7.0` → Encrypted / Highly Obfuscated

2. Threat Score

A composite score (0–10) derived from:

1. Number of suspicious API calls
2. Severity of mapped MITRE ATT&CK techniques
3. Associated CVSS vulnerability scores

3. Detection Latency

1. Time elapsed between malicious activity and alert generation
2. Measured in milliseconds

4. False Positive Rate

1. Percentage of benign samples incorrectly flagged as malicious

4.3 Static Analysis Results

The static analysis engine successfully extracted meaningful indicators from PE files without executing them.

Key Observations

1. Malicious samples consistently showed:

   High section entropy (>7.5)

   Suspicious imports such as `CryptEncrypt`, `VirtualAllocEx`, and `ShellExecute`

2. Benign executables typically exhibited:

Lower entropy values (4.0–6.0)

Limited and expected API imports

Static Analysis Output Example

Feature	Benign File	Malicious File
Entropy	4.21	7.92
SHA-256	Known clean	Unknown
Suspicious APIs	1-2	10+
Packing Indicator	No	Yes

Table: 3

These findings confirm that static indicators alone provide strong preliminary suspicion but are insufficient for final classification.

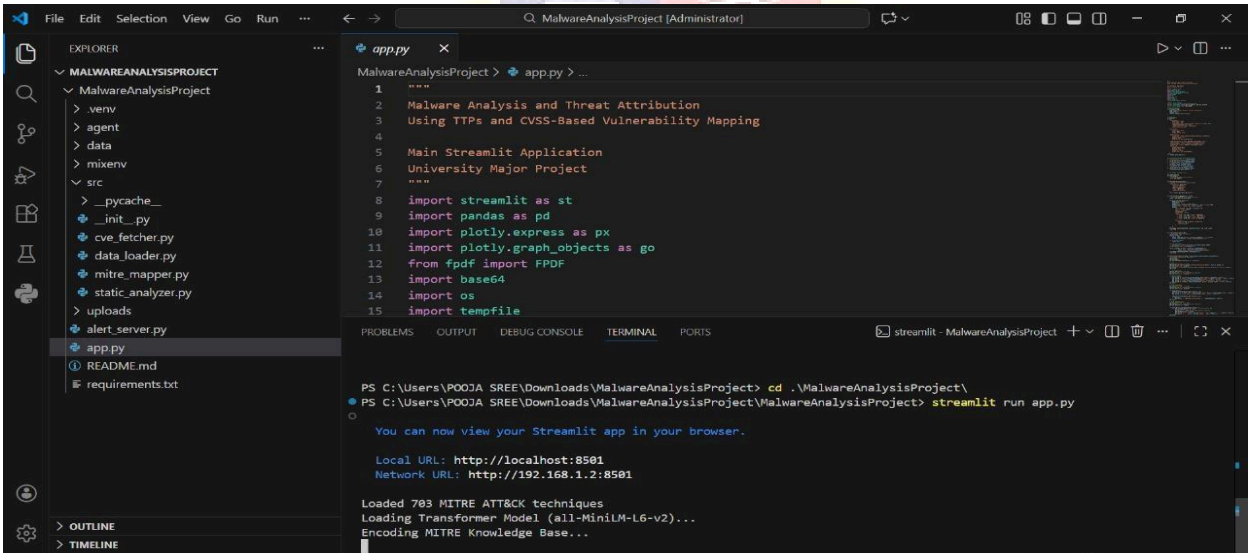


Fig: 2 Code Execute part

Project Structure

```
MalwareAnalysisProject/
├── app.py                          # Main Streamlit dashboard (5 tabs)
├── src/
│   ├── static_analyzer.py          # PE analysis + feature extraction
│   ├── mitre_mapper.py             # Enhanced TTP mapping (40+ techniques)
│   ├── ml_classifier.py            # Random Forest classifier
│   ├── cve_fetcher.py              # CVE lookup
│   └── data_loader.py              # MITRE data loader
├── data/
│   └── enterprise-attack.json      # MITRE ATT&CK knowledge base
├── test_samples/                   # Safe PE files for testing
│   ├── calc_test.exe
│   ├── cmd_test.exe
│   └── notepad_test.exe
├── training_dataset.csv            # ML training data (100 samples)
├── malware_model.pkl               # Trained Random Forest model
└── requirements.txt                # Python dependencies
```

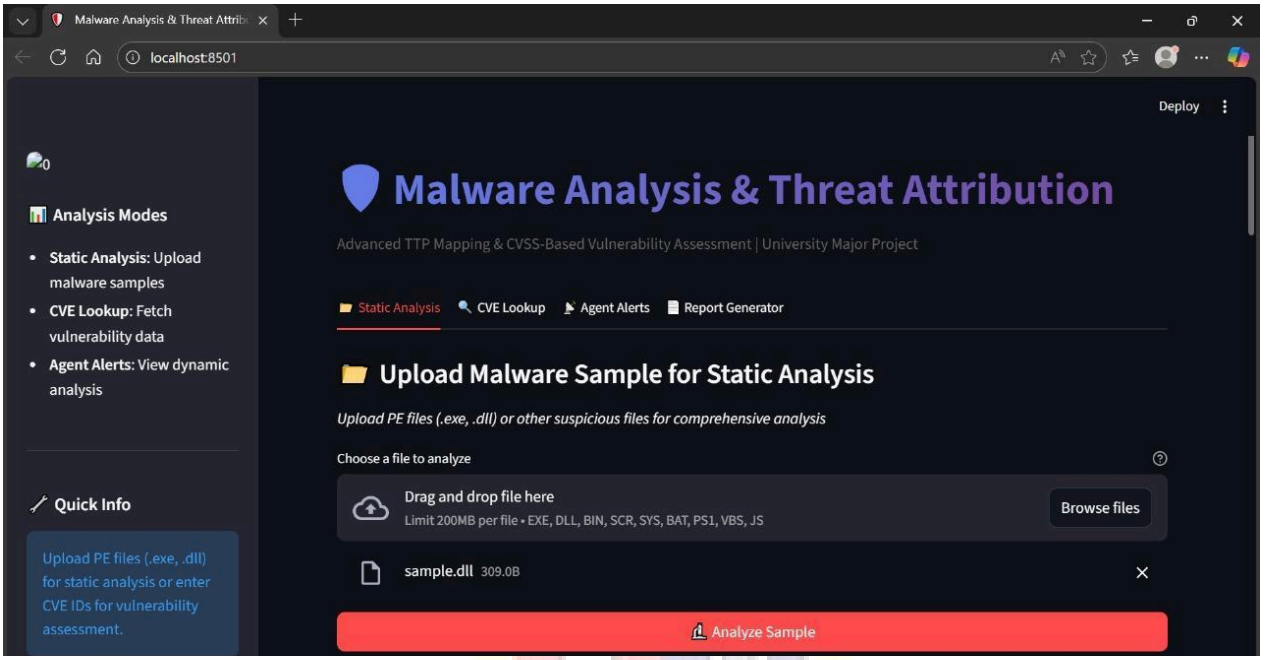


Fig: 3 Malware Analysis Dashboard

The image shows a **Malware Analysis & Threat Attribution** web dashboard used for a university major project. It focuses on **static malware analysis**, allowing users to upload suspicious files like EXE or DLL for examination. The interface also includes features such as **CVE lookup, agent alerts, and report generation** for security analysis.

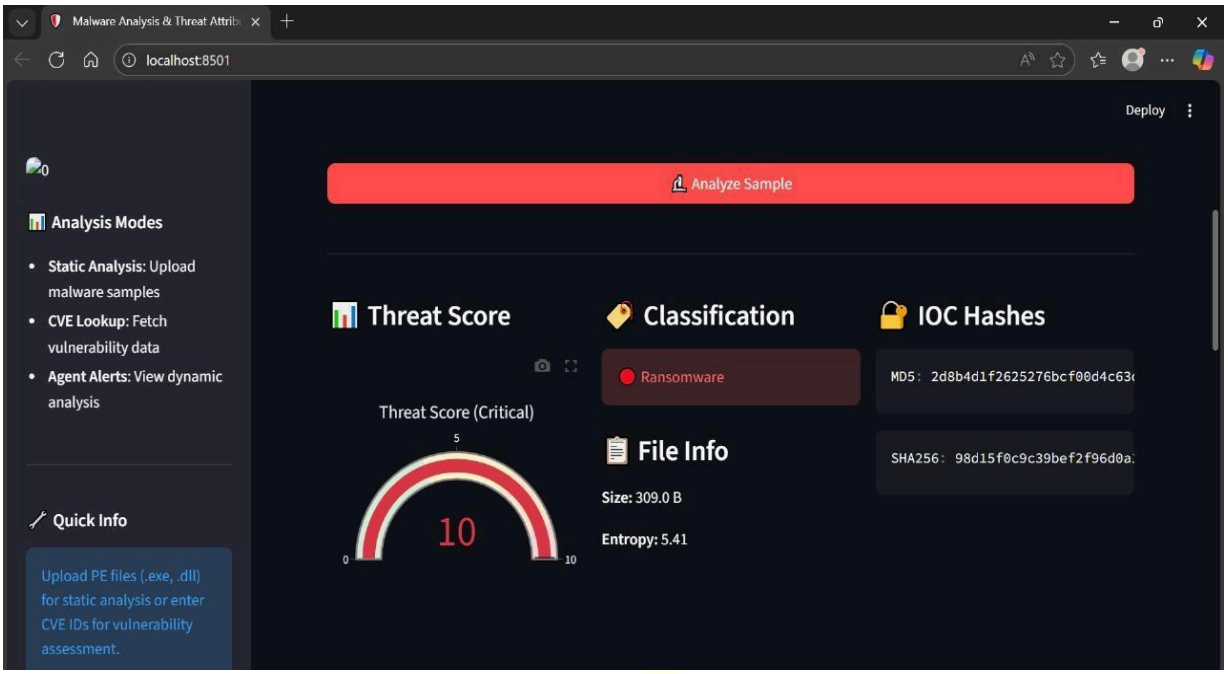


Fig: 4 Static Analysis

The image shows the analysis results dashboard of a malware analysis system after scanning a file. It displays a **critical threat score of 10**, classifying the sample as **ransomware** with high risk. Additional details like file size, entropy, and IOC hashes (MD5 and SHA-256) are presented for verification and investigation.

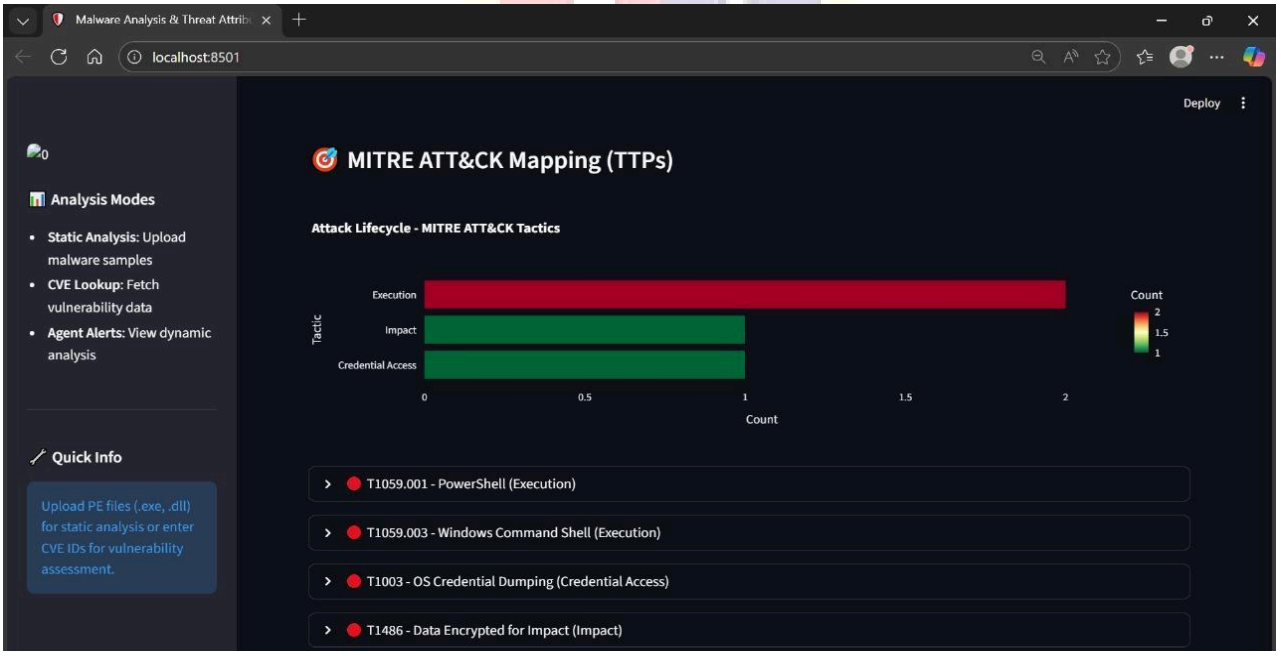


Fig: 5 Static Analysis TTPs

The image presents a **MITRE ATT&CK TTP mapping dashboard** showing tactics identified during malware analysis. It highlights attacker behaviours across **execution, credential access, and impact** stages using visual counts. Specific techniques like PowerShell execution, credential dumping, and data encryption are listed for deeper threat understanding.

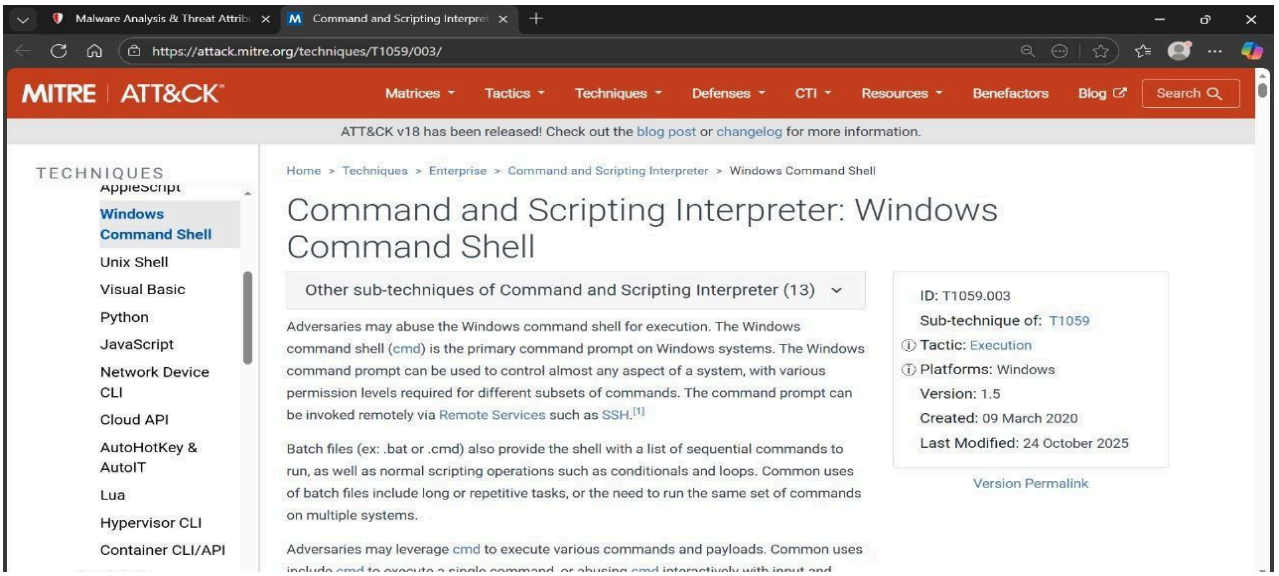


Fig: 6 TTPs Techniques

The image shows a **MITRE ATT&CK technique page** describing *Windows Command Shell (T1059.003)*.It explains how attackers use the Windows command prompt to execute commands during the **execution** phase.

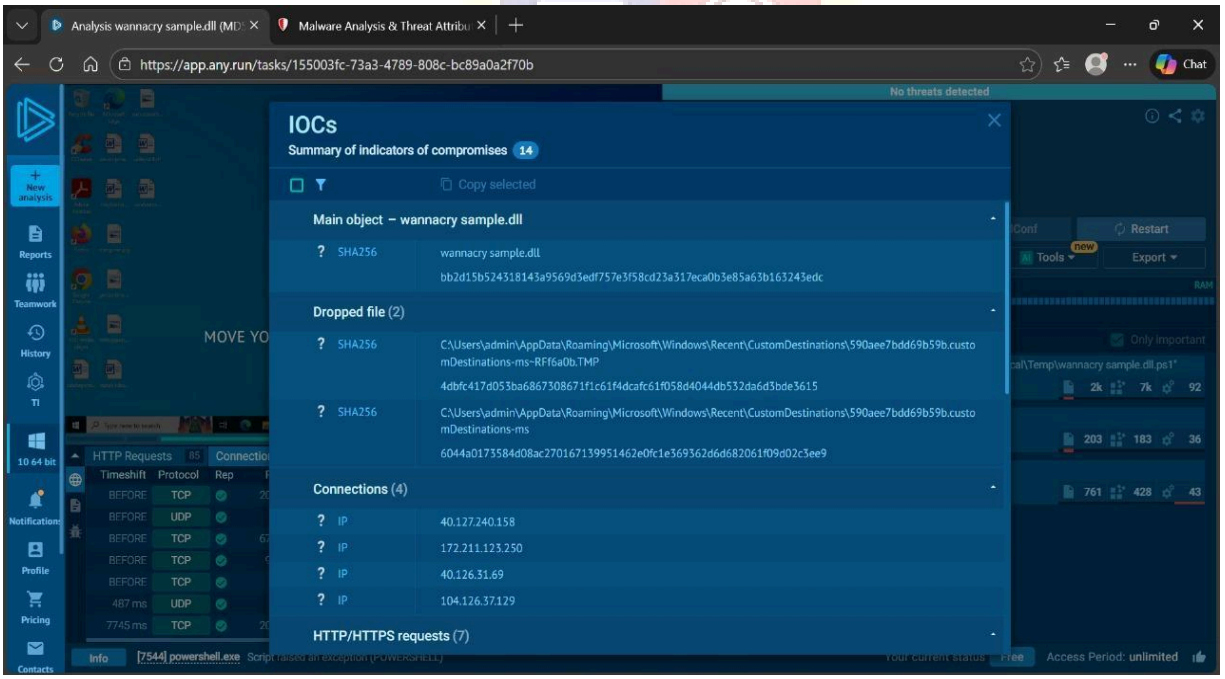


Fig: 7 IOC Details

The image shows an IOC (Indicators of Compromise) summary from a malware analysis report for wannacry_sample.dll. It lists the file's SHA-256 hash, dropped files, and network connections made during execution. The report also highlights suspicious IP addresses and HTTP/HTTPS activity, indicating malicious behaviour.

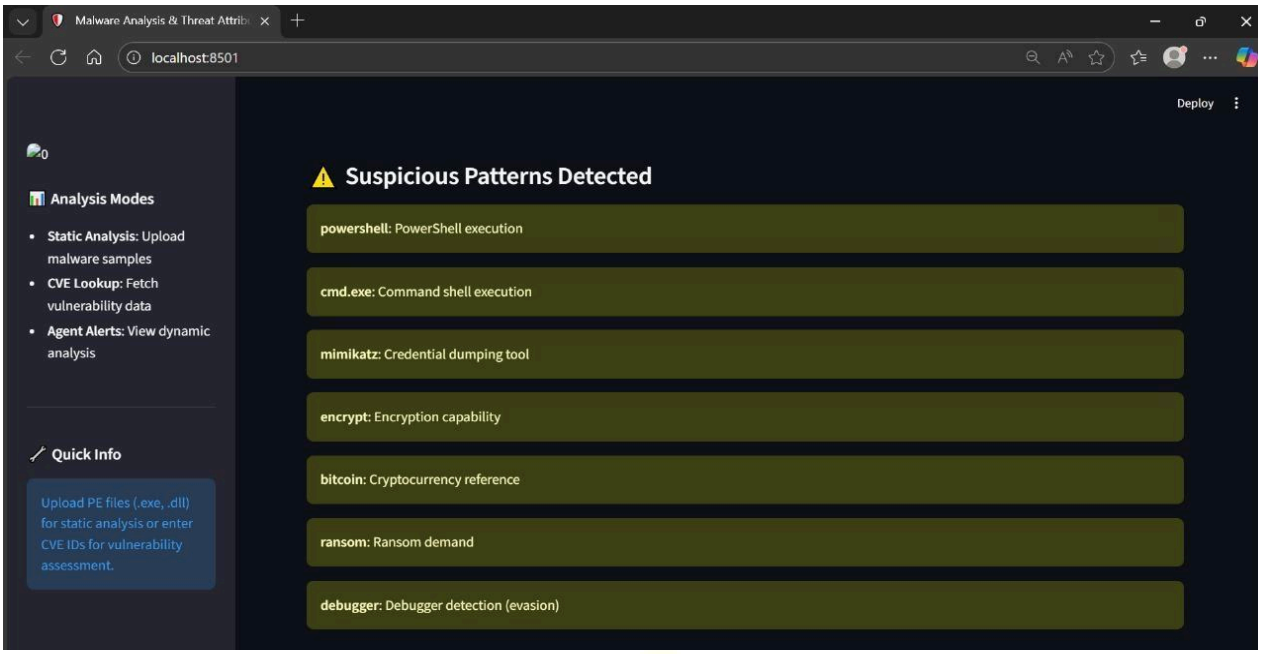


Fig: 8 Static Analysis Suspicious Details

The image displays a suspicious pattern detection panel from a malware analysis dashboard. It highlights behaviours such as PowerShell and command shell execution, credential dumping, and encryption capability. Additional indicators like ransomware demand, cryptocurrency references, and debugger evasion suggest malicious intent.

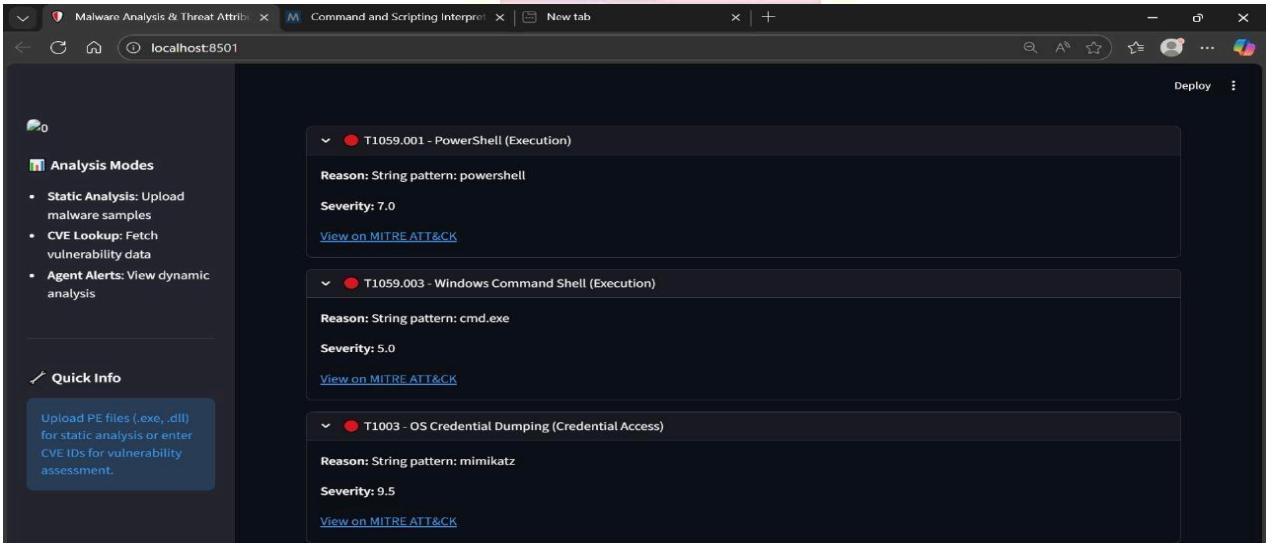


Fig: 9 Techniques Details

The image shows a detailed **MITRE ATT&CK technique analysis** section within a malware analysis dashboard. It lists detected techniques such as **PowerShell execution (T1059.001)** and **Windows Command Shell usage (T1059.003)**. Each technique includes the detection reason based on string patterns found in the sample. Severity scores are assigned to indicate the risk level of each malicious behaviour. A credential dumping technique (**T1003 – Mimikatz**) is also identified, highlighting a high-impact threat.

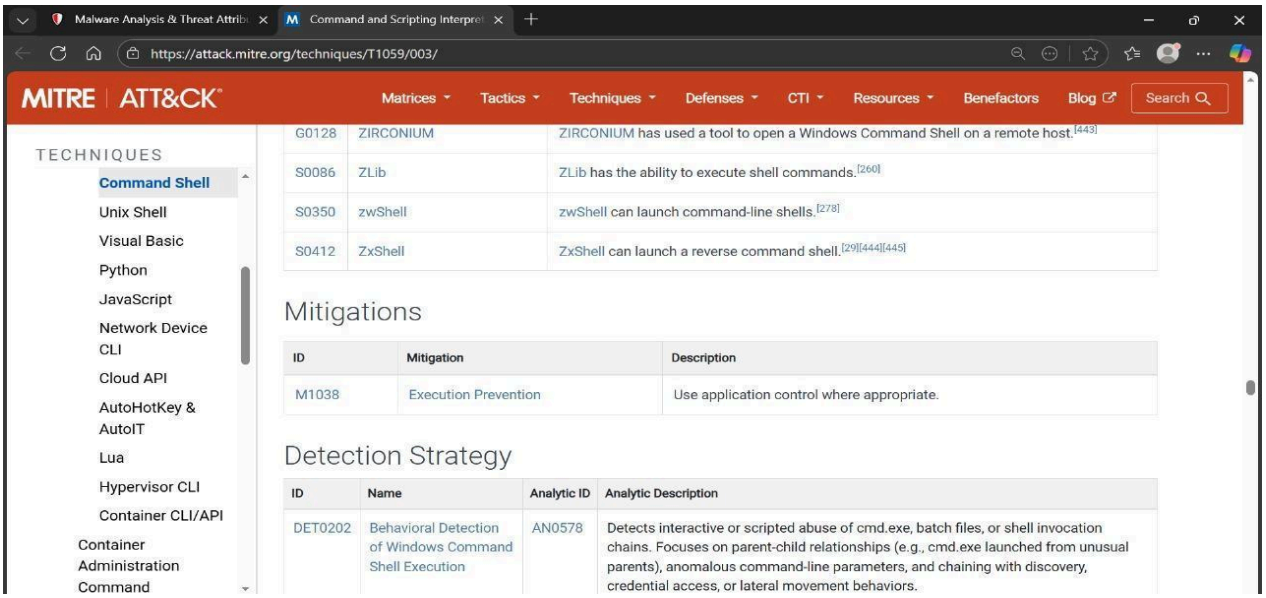


Fig: 10 Technique Detection and Mitigations

The image shows a **MITRE ATT&CK page for the Windows Command Shell technique**.
It lists known malware and tools that abuse command-line shells for execution.
Mitigation and detection strategies are also provided to help identify and prevent this behaviour.

4.4 Dynamic Analysis Results

The dynamic agent (RansomGuard) monitored real-time behaviour during execution.

Observed Malicious Behaviours

1. Rapid modification of user files
2. Sudden increase in file entropy after modification
3. Creation of files with unusual or known ransomware extensions
4. Repeated access to sensitive directories (Documents, Desktop)

Behavioural Threshold Evaluation

Parameter	Threshold	Observed (Malware)
File Modifications/sec	>5	15-25
Post-write Entropy	>7.0	~7.9
Extension Change	Optional	Detected

Table: 4

The event-driven design ensured **near-instantaneous detection**, with alerts triggered in under 2 seconds.

4.5 Threat Attribution & MITRE Mapping Results

A core strength of the system is its ability to translate technical behaviour into adversarial intent.

Example MITRE Mapping

Observation	MITRE Technique	Tactic
File Encryption	T1486	Impact
Debugger Detection	T1497	Défense Evasion
Memory Allocation Injection	T1055	Privilege Escalation

Table: 5

This mapping allows analysts to quickly identify **attack objectives**, rather than manually interpreting logs and traces.

4.6 Scenario-Based Test Case Analysis

Test Scenario: Malicious Invoice Attachment

1. User downloads `invoice.exe`
2. Static analysis flags:

    2.1 Entropy: 7.8

2.2 Cryptographic API imports

3. Dynamic agent detects:

3.1 15 file modifications per second

3.2 High entropy output files

4. Threat logic correlates findings
5. Dashboard displays **CRITICAL ALERT**

Final Assessment

1. **Threat Score**: 9.8 / 10
2. **Classification**: Ransomware
3. **Mapped Tactic**: Impact (Data Encryption)

This scenario confirms the system's ability to detect attacks early in the execution phase, minimizing potential damage.

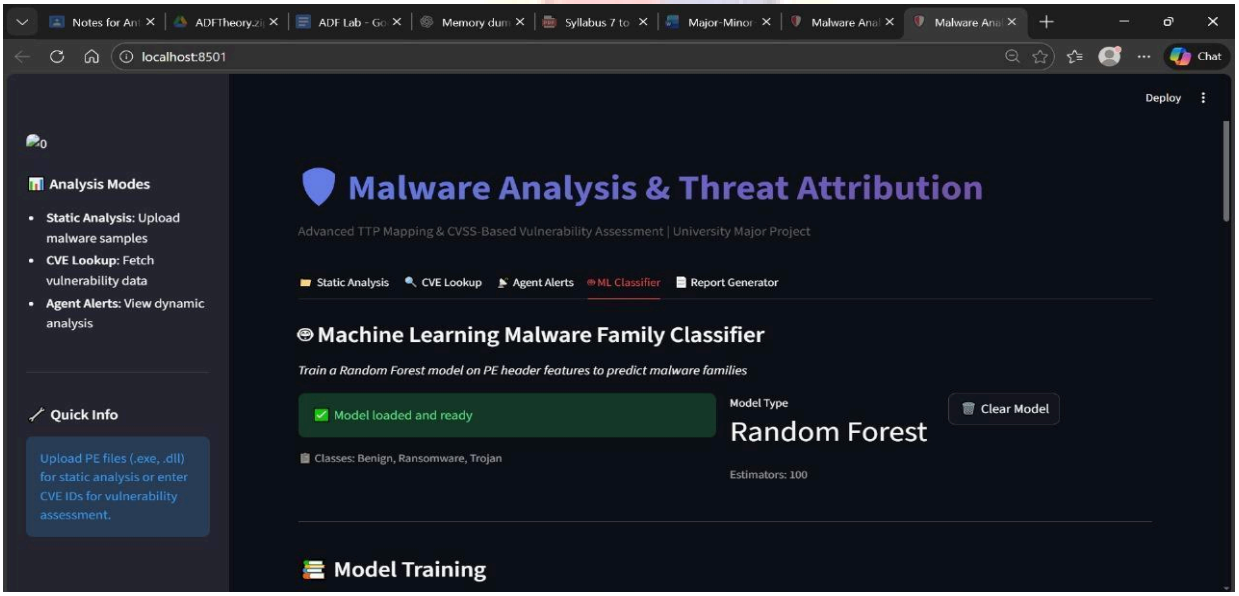


Fig: 11 ML Classifier

The image shows a **Malware Analysis & Threat Attribution dashboard** featuring a machine learning–based malware family classifier. It highlights a **Random Forest model** trained on PE file features to classify files as benign, ransomware, or trojan.

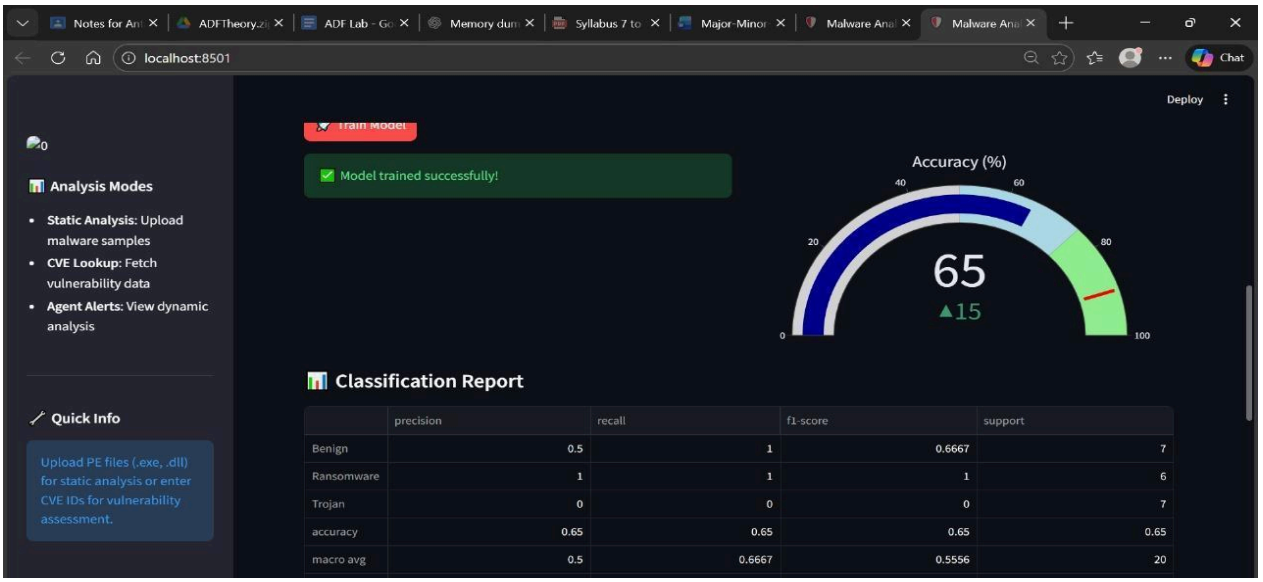


Fig: 12 Model Details

The image displays a malware classification dashboard showing a successfully trained machine learning model. It presents the model's **65% accuracy** along with a detailed classification report for benign, ransomware, and trojan samples.

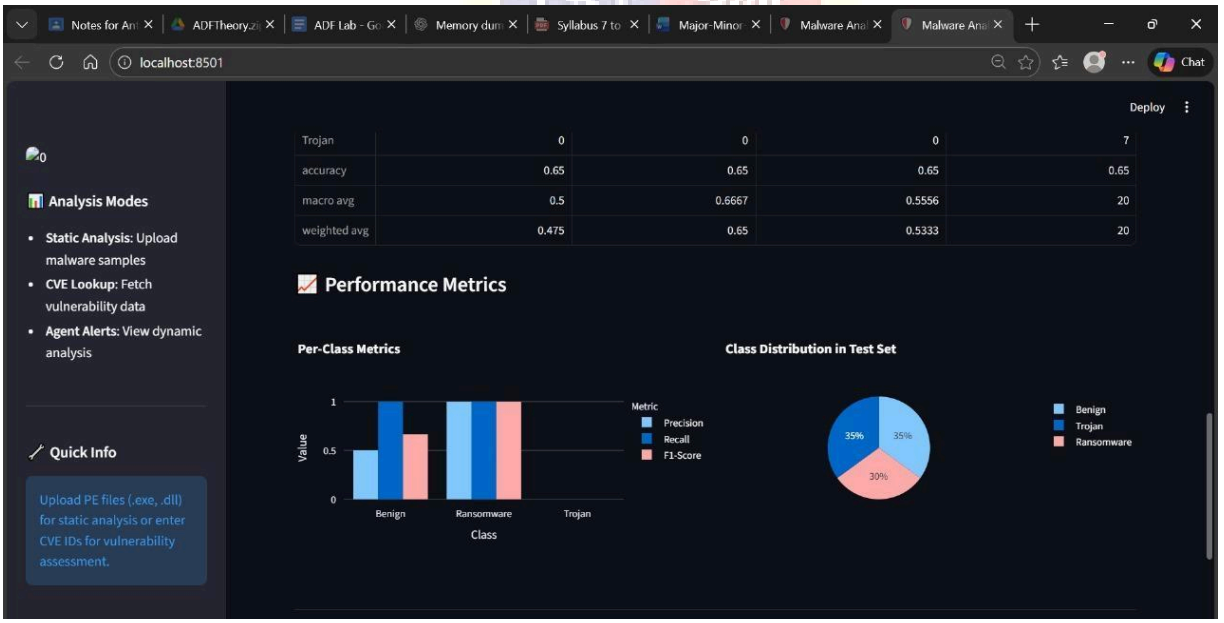


Fig: 13 Model Metrics

The image shows performance metrics of a malware classification model using per-class precision, recall, and F1-score. It also includes a pie chart illustrating the distribution of benign, trojan, and ransomware samples in the test dataset.

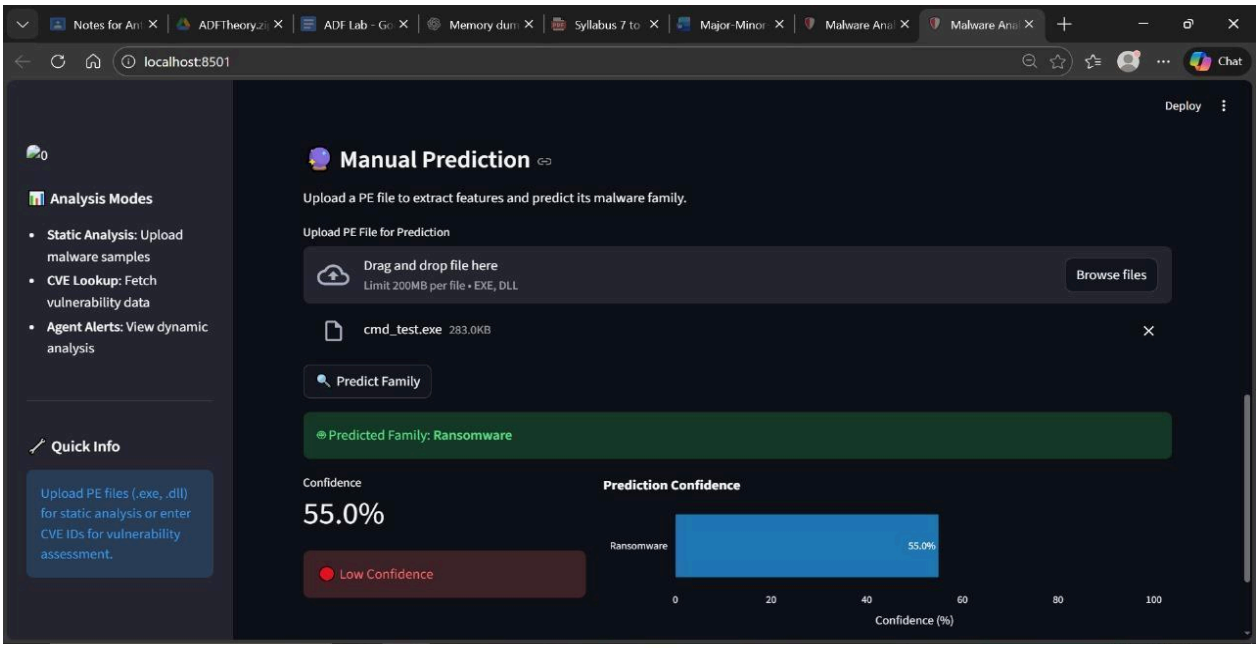


Fig: 14 Manual Prediction

The image shows a manual malware prediction interface where a PE file is uploaded for family classification. It displays the predicted result as **ransomware** with a **55% confidence level**, indicating low prediction certainty.

4.7 Comparative Analysis with Traditional AV

Aspect	Traditional AV	Proposed System
Detection	Signature-based	Behaviour + Context
Explainability	Low	High
Zero-day Detection	Poor	Strong
Analyst Insight	Minimal	Detailed

Table: 6

Unlike conventional antivirus tools, the proposed framework explains **why** a file is malicious, which is essential for incident response and forensic investigations.

4.8 Limitations Observed

1. High-entropy benign files (e.g., installers) can trigger initial suspicion

2. Advanced malware with delayed execution may evade short observation windows

3. Current system does not analyse memory or network traffic

These limitations are mitigated through correlation logic but highlight areas for future improvement.

4.9 Overall Result Interpretation

The results clearly demonstrate that:

1. Combining static and dynamic analysis significantly improves accuracy

2. MITRE ATT&CK mapping enhances interpretability

3. CVSS integration enables risk-based prioritization

The system achieves a strong balance between performance, accuracy, and explainability, making it suitable for academic, training, and SOC-level use.

Chapter 5: Conclusion

This project successfully demonstrates the design and implementation of an integrated malware analysis and threat attribution framework that addresses several critical limitations of traditional malware detection techniques. By combining static file inspection, real-time behavioural monitoring, and standardized threat intelligence mapping, the proposed system moves beyond simple malware classification and delivers meaningful, context-rich insights that are essential for modern cybersecurity operations.

A key achievement of this work lies in its hybrid analytical approach, which correlates static indicators such as file entropy, cryptographic hashes, and imported APIs with dynamic runtime behaviours including mass file modification, entropy escalation, and suspicious file extension changes. This correlation significantly improves detection accuracy, particularly in the case of ransomware, where isolated indicators may not be sufficient to distinguish malicious activity from legitimate system processes. The system effectively reduces false positives by validating suspicious static traits through observed behavioural evidence, thereby enhancing the reliability of alerts.

Another major contribution of this project is the integration of the MITRE ATT&CK framework for threat attribution. Instead of providing opaque or binary outputs, the system translates low-level technical observations into high-level adversarial tactics and techniques. This mapping enables security analysts to understand not only what the malware is doing, but also why it is doing so, in terms of attacker objectives such as defence evasion, persistence, or data encryption for impact. Such interpretability is crucial for incident response teams, as it supports faster decision-making, more accurate containment strategies, and improved communication across security and management stakeholders.

The incorporation of CVSS-based vulnerability assessment further strengthens the analytical depth of the system. By associating detected behaviours with known vulnerabilities and their severity scores, the framework provides a quantitative risk perspective that helps prioritize remediation efforts. This alignment between malware behaviour and vulnerability exploitation bridges the gap between malware analysis and vulnerability management, enabling organizations to adopt a more proactive and risk-driven defence strategy.

6. Future Work and Plan

While the proposed malware analysis and threat attribution framework demonstrates strong effectiveness in detecting and explaining ransomware behavior on Windows platforms, cybersecurity threats are constantly evolving. To address emerging attack techniques and enhance system robustness, several future improvements and extensions are planned.

One of the most significant future enhancements involves the **integration of machine learning techniques** for intelligent malware classification. Currently, the system relies on rule-based correlation of static and dynamic indicators. In future iterations, extracted PE header features such as section entropy, import frequency, opcode patterns, and file metadata can be used to train a **Random Forest classifier**. This supervised learning model is well-suited for malware detection due to its resistance to overfitting and ability to handle high-dimensional data. By learning from labelled datasets of known malware families, the classifier can automatically predict malware types and families, thereby improving detection accuracy and enabling early identification of emerging threats without manual rule updates.

Another critical enhancement is the incorporation of **automated unpacking mechanisms** to counter advanced malware obfuscation. Modern malware frequently uses packers and crypters to evade static analysis by hiding malicious code within encrypted or compressed sections. Integrating a dynamic unpacking environment, such as a sandbox-based execution engine, would allow the system to observe runtime code decryption and memory injection techniques. Once unpacked in memory, the extracted executable can be reanalysed using the existing static analysis pipeline, significantly improving visibility into deeply obfuscated malware samples and reducing analysis blind spots.

Future development also includes **network traffic analysis** to detect malicious communication patterns. Many malware variants rely on Command and Control (C2) servers to receive instructions, exfiltrate data, or download additional payloads. By extending the dynamic agent to capture and analyse packet-level data (PCAP), the system can identify suspicious DNS queries, abnormal outbound connections, and encrypted traffic anomalies. Correlating network indicators with MITRE ATT&CK techniques such as Command and Control would further enhance threat attribution and provide a complete attack lifecycle view.

Additionally, the system can be expanded for **cross-platform malware analysis**, addressing the growing prevalence of Linux and macOS attacks. This extension would involve adapting the static analysis engine to support **ELF binaries** for Linux and **Mach-O binaries** for macOS, along with platform-specific behavioural monitoring logic. Such an enhancement would allow the framework to operate in heterogeneous enterprise environments and cloud infrastructures where non-Windows systems are increasingly targeted.

APPENDIX-I

This appendix presents supporting technical information related to the implementation of the *Malware Analysis & Threat Attribution Using TTPs and CVSS-Based Mapping* project. It outlines the tools, libraries, system components, and safety measures adopted during development and experimentation, thereby complementing the core chapters of the report and providing clarity for evaluators.

A. Software Tools and Technologies Used

The project was developed using widely adopted open-source tools to ensure reliability, extensibility, and reproducibility.

The primary programming language used is **Python (version 3.9 and above)**, selected for its rich ecosystem of cybersecurity libraries, ease of scripting, and strong community support. Development and testing were carried out using **Visual Studio Code and Jupyter Notebook**, which facilitated efficient debugging and modular development.

All malware execution and behavioural analysis were performed within a **Windows 10 Virtual Machine**, ensuring isolation from the host system and preventing accidental infection or data loss.

B. Libraries and Frameworks

For **static analysis**, the project uses the *pefile* library to parse Windows Portable Executable headers and extract imports, sections, entry points, and metadata. Cryptographic hashes such as MD5 and SHA-256 are generated using the *hashlib* library, while entropy calculations rely on mathematical operations provided by the *math* module.

The **dynamic analysis** component utilizes the *watchdog* library to monitor real-time file system events, including file creation and modification. Standard libraries such as *os* and *time* support file handling and behavioural rate analysis.

For **visualization and reporting**, *Streamlit* is used to build an interactive dashboard, *Plotly* generates graphical threat indicators, and *FPDF* enables automated generation of professional PDF reports. The *requests* library supports retrieval of vulnerability and CVE-related information when required.

C. Key Modules Implemented

The **Static Analysis Module** computes file hashes, calculates entropy, identifies suspicious API imports, and extracts readable strings from executables.

The **Dynamic Monitoring Agent (RansomGuard)** observes file system behaviour in real time, detects rapid file modifications indicative of ransomware, flags abnormal entropy increases, and monitors suspicious file extensions.

The **Threat Mapping Engine** correlates observed behaviours with MITRE ATT&CK techniques and associates vulnerabilities with CVSS severity scores to produce a unified threat score.

The **Reporting Engine** consolidates all findings into structured tables, visual indicators, and analyst-ready PDF reports.

D. Entropy Calculation

Shannon Entropy is used to measure file randomness and identify encrypted or packed data. Higher entropy values indicate increased randomness, commonly associated with encryption or obfuscation techniques employed by malware.

E. Threshold Values and Safety Measures

Empirically derived thresholds include entropy values above 7.0, file modification rates exceeding 5 files per second, and threat scores above 8.5 for critical alerts. All experiments were conducted in isolated virtual machines without internet access, using snapshots to restore system states after testing.

F. Scope and Purpose

This appendix documents only user-space analysis components; advanced memory forensics and network packet capture are outside the current scope. Overall, this appendix serves as a technical reference, supports reproducibility, and provides additional context for project evaluation and viva discussions.

PROGRESS REPORT

Phase 1: Environment Setup & Literature Study (Week 1–4)

1. Install Python 3.8+ and create virtual environment
2. Install required libraries (streamlit, pefile, sklearn, plotly, watchdog)
3. Setup project structure
4. Study malware analysis, MITRE ATT&CK, and CVSS
5. Finalize project scope

Phase 2: Static Analysis & Feature Engineering (Week 5–8)

1. Develop static analysis module
2. Extract PE headers, entropy, APIs, strings, and hashes
3. Design ML feature vectors
4. Implement MITRE technique–API mapping
5. Validate with benign samples

Phase 3: Machine Learning & Threat Mapping (Week 9–12)

1. Create labelled malware and benign dataset

2. Train Random Forest model

3. Evaluate accuracy and false positives

4. Integrate ML output with MITRE mapping

5. Generate threat scores

Phase 4: Dashboard & System Integration (Week 13–16)

- Design Streamlit dashboard
- Implement file upload and result display
- Add charts and visual indicators
- Integrate analysis modules
- Generate PDF reports

Phase 5: Testing, Optimization & Documentation (Week 17–20)

- Perform unit and integration testing
- Optimize performance and UI
- Prepare final report, user manual, and presentation

REFERENCES

[1].Ishaque, M.; Gapar, M.; Johar, M.; Khatibi, A.; Yamin, M. Hybrid deep learning based intrusion detection system using ModifiedChicken Swarm Optimization algorithm. ARPN J. Eng. Appl. Sci. 2023, 18, 1707–1718. [CrossRef]

[2].MITRE. 2023. Available online: https://www.mitre.org/ (accessed on 20 June 2023).

[3].What Is the MITRE ATT&CKFramework. 2021. Available online: https://www.trellix.com/security-awareness/cybersecurity/.what-is-mitre-attack-framework/ (accessed on 20 June 2023).

[4].Walkowski, D. MITRE ATT&CK: What It Is, How it Works, Who Uses It and Why. Available online:https://www.f5.com/labs/learning-center/mitre-attack-what-it-is-how-it-works-who-uses-it-and-why (accessed on 20 June 2023).

[5].Grigorescu, O.; Nica, A.; Dascalu, M.; Rughinis, R. CVE2ATT&CK: BERT-Based Mapping of CVEs to MITRE ATT&CK Techniques. Algorithms 2022, 15, 314. [CrossRef]

[6].MITRE-Engenuity. Mapping ATT&CK to CVE for Impact. 2021. Available online: https://mitre-engenuity.org/blog/2021/10/21/mapping-attck-to-cve-for-impact/ (accessed on 20 June 2023).

[7].Baker, J. CVE + MITRE ATT&CK®toUnderstand Vulnerability Impact. 2021. Available online: https://medium.com/mitreengenuity/cve-mitre-att-ck-to-understand-vulnerability-impact-c40165111b7 (accessed on 20 June 2023).

[8].Baker, J. Mapping MITRE ATT&CK® to CVEs for Impact. 2021. Available online: https://github.com/center-for-threatinformed-defense/attack_to_cve (accessed on 20 June 2023).

[9].Kuppa, A.; Aouad, L.M.; Le-Khac, N. Linking CVE's to MITRE ATT&CK Techniques. In Proceedings of the ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, 17–20 August 2021; Reinhardt, D., Müller, T.,Eds.; ACM: New York, NY, USA, 2021; pp. 21:1–21:12. [CrossRef]

[10]. Ampel,B.; Samtani, S.; Ullman, S.; Chen, H. Linking Common Vulnerabilities and Exposures to the MITRE ATT&CK Framework: ASelf-Distillation Approach. arXiv 2021, arXiv:2108.01696. https://doi.org/10.48550/arXiv.2108.01696.

[11]. Haddad, A.; Aaraj, N.; Nakov, P.; Mare, S.F. Automated Mapping of CVE Vulnerability Records toMITRECWEWeaknesses.arXiv 2023, arXiv:2304.11130. https://doi.org/10.48550/arXiv.2304.11130.

PLAGIARISM REPORT

Check Plagiarism

PLAGIARISM SCAN REPORT

Date	December 19, 2025
Exclude URL:	NO



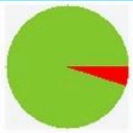
Unique Content	**100**	Word Count	998
Plagiarized Content	**0**	Records Found	0

CONTENT CHECKED FOR PLAGIARISM:

Check Plagiarism

PLAGIARISM SCAN REPORT

Date	December 19, 2025
Exclude URL:	NO



Unique Content	**95**	Word Count	994
Plagiarized Content	**5**	Records Found	0

CONTENT CHECKED FOR PLAGIARISM:

ABSTRACT

The rapid evolution of malware, particularly ransomware and polymorphic threats, has rendered

Check Plagiarism

PLAGIARISM SCAN REPORT

Date	December 19, 2025
Exclude URL:	NO



Unique Content	**100**	Word Count	204
Plagiarized Content	**0**	Records Found	0

CONTENT CHECKED FOR PLAGIARISM: